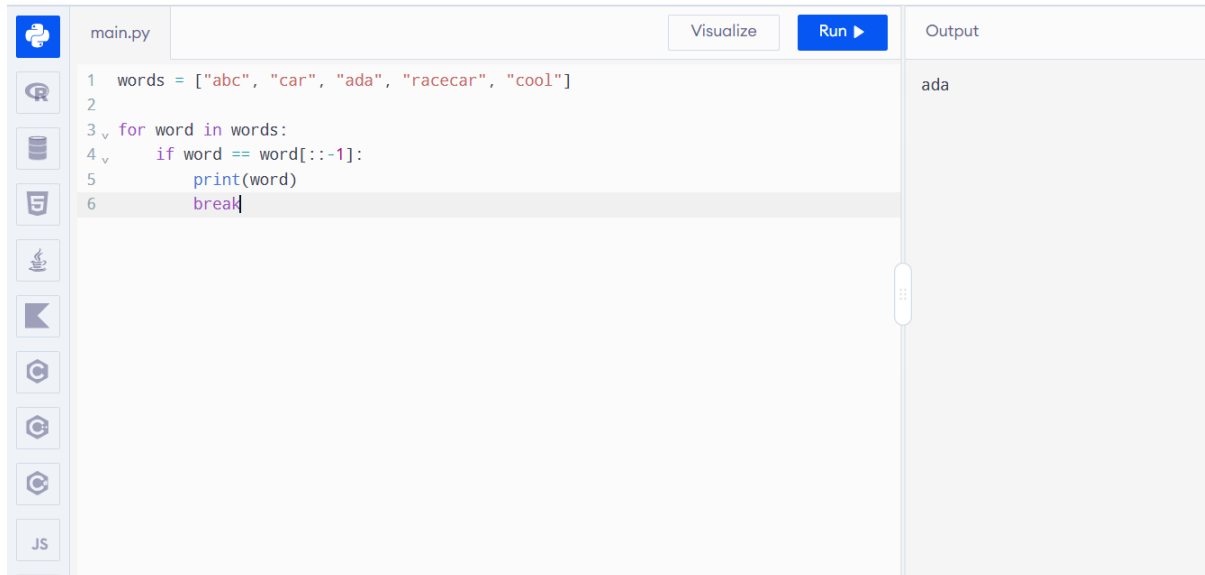


TOPIC 1 : INTRODUCTION

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.



The screenshot shows a Python IDE with a file named 'main.py'. The code is as follows:

```
1 words = ["abc", "car", "ada", "racecar", "cool"]
2
3 for word in words:
4     if word == word[::-1]:
5         print(word)
6         break
```

The 'Output' pane on the right shows the result: 'ada'.

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].



The screenshot shows a Python IDE with a file named 'main.py'. The code is as follows:

```
1 def find_intersection_values(nums1, nums2):
2     answer1 = 0
3     answer2 = 0
4
5     for x in nums1:
6         if x in nums2:
7             answer1 += 1
8
9     for x in nums2:
10        if x in nums1:
11            answer2 += 1
12
13    return [answer1, answer2]
14
15
16 # Input
17 nums1 = list(map(int, input("Enter nums1: ").split()))
18 nums2 = list(map(int, input("Enter nums2: ").split()))
19
20 # Output
21 print(find_intersection_values(nums1, nums2))
```

The 'Output' pane on the right shows the input and output: 'Enter nums1: 2 3 2', 'Enter nums2: 1 2', and '[2, 1]'.

3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'sum_of_squares' that takes a list 'nums' and returns the sum of squares of its distinct elements. The main part of the program takes user input for 'Enter nums: ' and prints the result of 'sum_of_squares'.

```

1 def sum_of_squares(nums):
2     total = 0
3
4     for i in range(len(nums)):
5         distinct = set()
6
7         for j in range(i, len(nums)):
8             distinct.add(nums[j])
9             count = len(distinct)
10            total += count * count
11
12    return total
13
14
15 nums = list(map(int, input("Enter nums: ").split()))
16
17 print(sum_of_squares(nums))

```

The output panel shows the input 'Enter nums: 1 2 1' and the output '15'.

4. Given a 0-indexed integer array *nums* of length *n* and an integer *k*, return *the number of pairs (i, j) where $0 \leq i < j < n$, such that $nums[i] == nums[j]$ and $(i * j)$ is divisible by k .*

The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'count_pairs' that takes a list 'nums' and an integer 'k', and returns the number of pairs (i, j) such that $nums[i] == nums[j]$ and $(i * j) \% k == 0$. The main part of the program takes user input for 'Enter nums: ' and 'Enter k: ', and prints the result of 'count_pairs'.

```

1 def count_pairs(nums, k):
2     count = 0
3
4     for i in range(len(nums)):
5         for j in range(i + 1, len(nums)):
6             if nums[i] == nums[j] and (i * j) % k == 0:
7                 count += 1
8
9     return count
10
11
12 nums = list(map(int, input("Enter nums: ").split()))
13 k = int(input("Enter k: "))
14
15 print(count_pairs(nums, k))

```

The output panel shows the input 'Enter nums: 1 2 2 3 3 4 5 6' and 'Enter k: 2', and the output '2'.

5. Write a program FOR THE BELOW TEST CASES with least time complexity
Test Cases: -

The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

```

1 nums = list(map(int, input("Enter numbers: ").split()))
2
3 maximum = nums[0]
4
5 for x in nums:
6     if x > maximum:
7         maximum = x
8
9 print(maximum)

```

The 'Output' pane on the right shows the result of running the code:

```

Enter numbers: 1 2 3 4 5
5

```

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

```

1 def find_maximum(nums):
2     if len(nums) == 0:
3         return None
4
5     nums.sort()
6     return nums[-1]
7
8
9 nums = list(map(int, input("Enter numbers: ").split()))
10
11 result = find_maximum(nums)
12
13 if result is None:
14     print("The list is empty")
15 else:
16     print(result)

```

The 'Output' pane on the right shows the result of running the code:

```

Enter numbers: 4 5 7 2 3 1
7

```

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

The screenshot shows a Python IDE with a file named 'main.py'. The code in the editor is as follows:

```

1 def unique_elements(nums):
2     unique = []
3
4     for x in nums:
5         if x not in unique:
6             unique.append(x)
7
8     return unique
9
10
11 nums = list(map(int, input("Enter numbers: ").split()))
12
13 print(unique_elements(nums))

```

The 'Output' pane on the right shows the result of running the code:

```

Enter numbers: 1 2 5 7 1 9 3 2
[1, 2, 5, 7, 9, 3]

```

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code



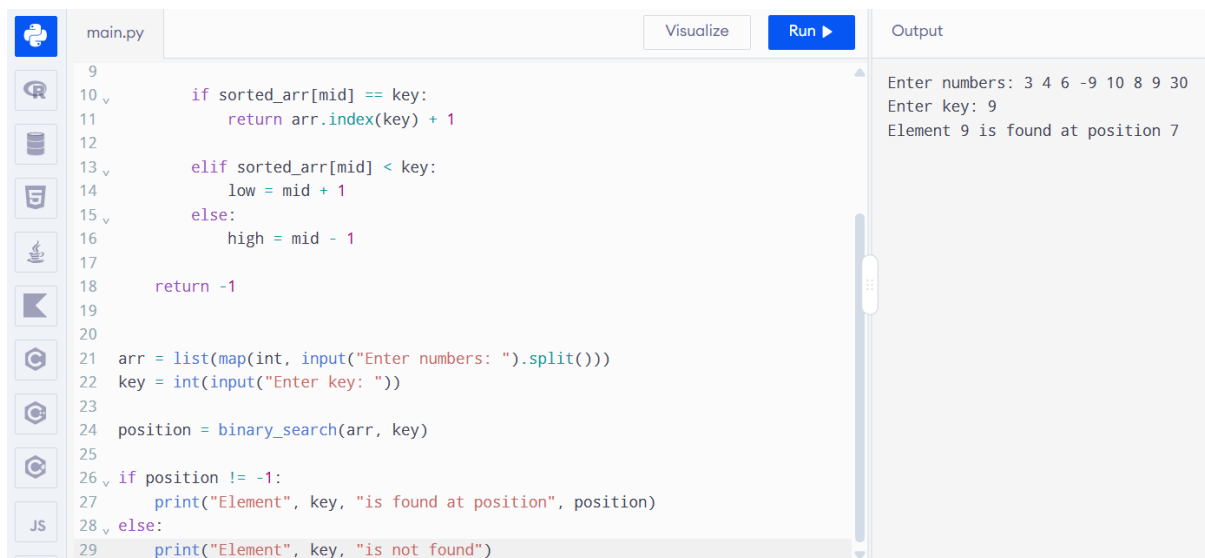
The screenshot shows a Python IDE with a file named 'main.py'. The code implements a bubble sort function. The output panel shows the input numbers and the sorted array.

```
1 def bubble_sort(nums):
2     n = len(nums)
3
4     for i in range(n):
5         for j in range(0, n - i - 1):
6             if nums[j] > nums[j + 1]:
7                 nums[j], nums[j + 1] = nums[j + 1], nums[j]
8
9     return nums
10
11
12 nums = list(map(int, input("Enter numbers: ").split()))
13
14 print("Sorted array:", bubble_sort(nums))
```

Output:

```
Enter numbers: 90 60 80 70 40 10
Sorted array: [10, 40, 60, 70, 80, 90]
```

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a binary search function. The output panel shows the input numbers, the key, and the position of the element.

```
9
10 if sorted_arr[mid] == key:
11     return arr.index(key) + 1
12
13 elif sorted_arr[mid] < key:
14     low = mid + 1
15 else:
16     high = mid - 1
17
18 return -1
19
20
21 arr = list(map(int, input("Enter numbers: ").split()))
22 key = int(input("Enter key: "))
23
24 position = binary_search(arr, key)
25
26 if position != -1:
27     print("Element", key, "is found at position", position)
28 else:
29     print("Element", key, "is not found")
```

Output:

```
Enter numbers: 3 4 6 -9 10 8 9 30
Enter key: 9
Element 9 is found at position 7
```

10. Given an array of integers $nums$, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a merge sort function. The output panel shows the input numbers and the sorted array.

```
1 def merge_sort(nums):
2     if len(nums) <= 1:
3         return nums
4
5     mid = len(nums) // 2
6
7     left = merge_sort(nums[:mid])
8     right = merge_sort(nums[mid:])
9
10    result = []
11    i = 0
12    j = 0
13
14    while i < len(left) and j < len(right):
15        if left[i] <= right[j]:
16            result.append(left[i])
17            i += 1
18        else:
19            result.append(right[j])
20            j += 1
```

Output:

```
Enter numbers: 4 5 6 1 2 3
Sorted array: [1, 2, 3, 4, 5, 6]
```

11. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.



```
main.py Visualize Run ▶ Output
30         else:
31             new_dp[r][c - 1] += dp[r][c]
32
33         # Right
34         if c == n - 1:
35             result += dp[r][c]
36         else:
37             new_dp[r][c + 1] += dp[r][c]
38
39         dp = new_dp
40
41     return result
42
43
44 m = int(input("Enter m: "))
45 n = int(input("Enter n: "))
46 N = int(input("Enter N: "))
47 i = int(input("Enter starting row i: "))
48 j = int(input("Enter starting column j: "))
49
50 print("Number of ways:", find_paths(m, n, N, i, j))
```

Enter m: 4
Enter n: 3
Enter N: 1
Enter starting row i: 1
Enter starting column j: 0
Number of ways: 1

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.



```
main.py Visualize Run ▶ Output
1 def rob_houses(nums):
2     if len(nums) == 1:
3         return nums[0]
4
5     def rob(arr):
6         prev = 0
7         curr = 0
8
9         for money in arr:
10             new = max(curr, prev + money)
11             prev = curr
12             curr = new
13
14         return curr
15
16     # Case 1: Don't rob the first house
17     case1 = rob(nums[1:])
18
19     # Case 2: Don't rob the last house
20     case2 = rob(nums[:-1])
21
```

Enter money in houses: 4 2 3 1 7
Maximum money: 10

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

```

main.py
Visualize Run ▶
Output
Enter n: 4
Number of ways: 5

1 def climb_stairs(n):
2     if n <= 2:
3         return n
4
5     a = 1
6     b = 2
7
8     for i in range(3, n + 1):
9         c = a + b
10        a = b
11        b = c
12
13    return b
14
15
16 n = int(input("Enter n: "))
17
18 print("Number of ways:", climb_stairs(n))

```

14. A robot is located at the top-left corner of a $m \times n$ grid. The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

```

main.py
Visualize Run ▶
Output
Enter m: 4
Enter n: 3
Number of unique paths: 10

1 def unique_paths(m, n):
2     dp = [1] * n
3
4     for i in range(1, m):
5         for j in range(1, n):
6             dp[j] = dp[j] + dp[j - 1]
7
8     return dp[n - 1]
9
10
11 m = int(input("Enter m: "))
12 n = int(input("Enter n: "))
13
14 print("Number of unique paths:", unique_paths(m, n))

```

15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like $s = \text{"abbxxxxzzy"}$ has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval $[\text{start}, \text{end}]$, where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval $[3, 6]$. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.



The screenshot shows a Jupyter Notebook with a file named 'main.py'. The code defines a function 'large_groups(s)' that iterates through a string 's' and identifies groups of identical characters. The function returns a list of lists, where each inner list contains the start and end indices of a group. The output shows the input string 'abbxxxxzzy' and the resulting list of groups '[[3, 6]]'.

```
1 def large_groups(s):
2     result = []
3     start = 0
4
5     for i in range(1, len(s) + 1):
6         if i == len(s) or s[i] != s[start]:
7             if i - start >= 3:
8                 result.append([start, i - 1])
9
10        start = i
11
12    return result
13
14
15 s = input("Enter string: ")
16
17 print(large_groups(s))
```

Output: Enter string: abbxxxxzzy
[[3, 6]]

16. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an $m \times n$ grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

- Any live cell with fewer than two live neighbors dies as if caused by under-population.

- Any live cell with two or three live neighbors lives on to the next generation.

- Any live cell with more than three live neighbors dies, as if by over-population.

- Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the $m \times n$ grid board, return *the next state*.

main.py

Visualize

Run ▶

```

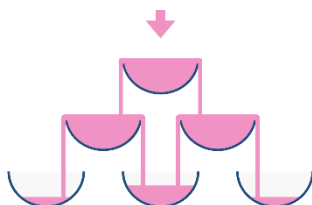
1 def game_of_life(board):
2     m = len(board)
3     n = len(board[0])
4
5     new_board = [[0] * n for _ in range(m)]
6
7     for i in range(m):
8         for j in range(n):
9             live = 0
10
11             # Check all 8 neighbors
12             for di in [-1, 0, 1]:
13                 for dj in [-1, 0, 1]:
14                     if di == 0 and dj == 0:
15                         continue
16
17                     ni = i + di
18                     nj = j + dj
19
20                     if 0 <= ni < m and 0 <= nj < n:
21                         live += board[ni][nj]

```

Output

Enter rows: 4
Enter columns: 3
Enter the board rows:
0 1 0
0 0 1
1 1 1
0 0 0
Next state:
[0, 0, 0]
[1, 0, 1]
[0, 1, 1]
[0, 1, 0]

17. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.



Now after pouring some non-negative integer cups of champagne, return how full the j^{th} glass in the i^{th} row is (both i and j are 0-indexed.)



main.py

Visualize

Run ▶



3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

```
glasses[0][0] = poured

for i in range(99):
    for j in range(i + 1):
        if glasses[i][j] > 1:
            excess = (glasses[i][j] - 1) / 2

            glasses[i + 1][j] += excess
            glasses[i + 1][j + 1] += excess

            glasses[i][j] = 1

return min(1, glasses[query_row][query_glass])

poured = int(input("Enter poured: "))
query_row = int(input("Enter query row: "))
query_glass = int(input("Enter query glass: "))

print("Amount in glass:", champagne_tower(poured, query_row, query_glass))
```

Output

Enter poured: 2
Enter query row: 1
Enter query glass: 1
Amount in glass: 0.5